

Reviewer Pack — Minimal Number of Generators for Affine Quotient Groups and ...

Entry 02e01f993beb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 14:43:58 UTC

Minimal Number of Generators for Affine Quotient Groups and Circuit Monotone Width

Entry ID: 02e01f993beb **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 14:43:58 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For any Boolean circuit C with n inputs and depth d , the minimal number of generators required to construct the affine quotient group associated with C is upper-bounded by $O(n^{(1/2)^d}(3/2))$.

Rationale (proposer's reasoning):

Affine geometry provides a natural way to encode the structure of Boolean circuits, particularly through the study of affine quotient groups. A stronger understanding of these groups could potentially reveal deep connections between geometric properties and computational complexity.

Taxonomy category: affine_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1484be9c8453f1ab

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given Boolean circuit C , if the monotone width of C is less than or equal to $O(n^{(1/2)^d}(3/2))$ with at least 80% of the seeds producing a generator count within this bound, then the conjecture is supported. Falsification occurs if any seed produces a generator count greater than $O(n^{(1/2)^d}(3/2))$ or the monotone width exceeds this upper bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine geometry" AND "Boolean circuit complexity" AND "minimal number of generators" - "affine quotient group" AND circuit monotone width AND "Affine Geometry" - "circuit depth" AND generator bound AND "affine geometry"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_circuit(n, d):
    if n == 1:
        return [random.choice([0, 1])]
    elif d == 2:
        inputs = generate_boolean_circuit(n // 2, d - 1)
        return [random.choice([0, 1]) for _ in range(2 * len(inputs))]
    else:
        left = generate_boolean_circuit(n // 2, d - 1)
        right = generate_boolean_circuit(n // 2, d - 1)
        return [random.choice(left + right) for _ in range(len(left))]

def compute_monotone_width(circuit):
    n = len(circuit)
    width = 0
    for i in range(1 << n):
        active = [j for j in range(n) if (i >> j) & 1]
        if all(circuit[j] == circuit[active[0]] for j in active):
            width = max(width, len(active))
    return width

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
```

```

for _ in range(5): # Ensure at least 30 instances per seed
    circuit = generate_boolean_circuit(n, random.randint(2, 5))
    monotone_width = compute_monotone_width(circuit)
    upper_bound = math.ceil(math.sqrt(n) * (random.randint(2, 5) ** (3/2)))
    if monotone_width > upper_bound:
        conjecture_holds = False
        counterexample = f"n={n}, d={len(circuit)}, width={monotone_width}, bound={upper_bound}"
        break
    total_metric_value += monotone_width
    instances_tested += 1

return {
    "metric_name": "Monotone Width",
    "metric_value": total_metric_value / instances_tested,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

True, 'counterexample': ''
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.4333333333333333, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.5, 'instances_tested': 30, 'n_max': 40, 'c
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.5666666666666667, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.3666666666666667, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.5666666666666667, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.3, 'instances_tested': 30, 'n_max': 40, 'c
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.6666666666666667, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.5666666666666667, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.6, 'instances_tested': 30, 'n_max': 40, 'c

```

```

TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.6, 'instances_tested': 30, 'n_max': 40, 'c
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.4666666666666666, 'instances_tested': 30,
TRIAL: {'metric_name': 'Monotone Width', 'metric_value': 1.5333333333333334, 'instances_tested': 30,
RESULT: SUPPORTED mean=1.5055555555555555 std=0.09312808119022341 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers a small range of circuit depths (2 to 5) and does not scale with n , which may not be representative of the conjecture's claim for circuits of arbitrary depth. Additionally, the upper bound is randomly chosen within a fixed range, which could lead to an overestimation of the monotone width.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results show that for all tested instances within the specified range of circuit depths (2 to 5), the monotone width and the number of genera | next: Further testing with a wider range of circuit depths and varying n values to validate the conjecture's claim for circuits of arbitrary depth.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16495
2	propose	ollama_remote	glm4:latest	0	0	15003
3	preregistration	ollama_remote	glm4:latest	0	0	13008
4	novelty	ollama_remote	glm4:latest	0	0	8787
5	novelty	ollama_remote	glm4:latest	0	0	17652
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14436
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11278
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9507
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12358
10	critic	ollama_remote	glm4:latest	0	0	14655
11	judge	ollama_remote	glm4:latest	0	0	14066

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147246 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/02e01f993beb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/02e01f993beb.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/02e01f993beb.tar.gz` (if generated)